

P3423R0

Extending User-Generated Diagnostic Messages

Published Proposal, 2024-10-15

This version:

<https://wg21.link/P3423R0>

Author:

[Yihe Li](#)

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Target:

C++26

Source:

github.com/Mick235711/wg21-papers/blob/main/P3423/P3423R0.bs

Issue Tracking:

[GitHub Mick235711/wg21-papers](#)

Table of Contents

1 Revision History

1.1 R0 (2024-10 pre-Wrocław Mailing)

2 Motivation and History

3 Usage Example

3.1 Standard Library

3.2 Third-Party Libraries

4 Design

- 4.1 What Is A String?
- 4.2 Design Decisions
- 4.3 Attribute Expression Argument
- 4.4 Attribute Placement and Name Visibility
- 4.5 What If The Message Parameter Is Invalid?
- 4.6 Alternative Syntax Considered
- 4.7 Claim of Syntax Space
- 4.8 Interaction with Reflection
- 4.9 Proposal Scope
- 4.10 Target Vehicle
- 4.11 Feature Test Macro

5 Implementation Experience

6 Wording

- 6.1 7.7 Constant expressions [expr.const]
- 6.2 9.1 Preamble [dcl.pre]
- 6.3 9.5 Function definitions [dcl.fct.def]
 - 6.3.1 9.5.1 General [dcl.fct.def.general]
 - 6.3.2 9.5.3 Deleted definitions [dcl.fct.def.delete]
- 6.4 9.12 Attributes [dcl.attr]
 - 6.4.1 9.12.5 Deprecated attribute [dcl.attr.deprecated]
 - 6.4.2 9.12.10 Nodiscard attribute [dcl.attr.nodiscard]
- 6.5 15.2 Conditional inclusion [cpp.cond]
- 6.6 15.11 Predefined macro names [cpp.predefined]

References

Normative References

Informative References

During Varna (2023-06), [\[P2741R3\]](#) had been adopted into the C++26 working draft, which gave `static_assert` the ability to accept a user-generated string-like object as the message parameter. This extension allowed the user of `static_assert` to provide a more precise error message in compile time, thus significantly increasing the user-friendliness of libraries. This proposal, therefore, proposes to unify the language by allowing other constructs in the language that currently accept a mes-

sage parameter, namely `[[nodiscard]]`, `[[deprecated]]`, and `= delete`, to also allow a user-generated string-like object as the provided message.

§ 1. Revision History

§ 1.1. R0 (2024-10 pre-Wrocław Mailing)

- Initial revision.

§ 2. Motivation and History

The first appearance of the compile-time message parameter in the C++ core language is in C++11, where `static_assert` declaration was introduced into the language by [N1720] with the syntax `static_assert(condition, message)`. At that time, the message parameter was the first time that users of the C++ language were able to control the generation of diagnostic messages in a dynamic way (excluding the preprocess macro `#error`, which is not controllable by the language itself, let alone dynamically in compile-time programming). Such flexibility has allowed the emergence of user-friendly compile-time libraries, where error messages can directly present to the user what they have done wrong. For instance, the following misuse of `std::print`:

```
#include <print>

struct A {};

int main()
{
    A a;
    std::println("{} ", a);
}
```

can directly generate user-friendly diagnostics under clang 19.1.0 with libstdc++:

```
[...]  
In file included from <source>:1:  
In file included from /opt/compiler-explorer/gcc-14.2.0/lib/gcc/x86_64-linux-  
/opt/compiler-explorer/gcc-14.2.0/lib/gcc/x86_64-linux-gnu/14.2.0/../../../../...
```

```
3732 |         static_assert(is_default_constructible_v<formatter<_Tq, _  
    |  
[...]
```

Although a bit cramped and hidden, the error message directly tells the user that `std::formatter` must be specialized, thus informing them how to fix the problem. Without this feature, the user will only be faced with cryptic error messages such as "`std::formatter<A, char> is not default constructible`", which requires an expert with a deep understanding of the specification of `std::formatter` to uncover the actual cause of the error.

Over the following years, more and more control over the generation and content of the diagnostic message of the compiler had been opened to the library writer:

- In C++14, [N3760] introduced the `[[deprecated]]` attribute with an optional message parameter.
- In C++17, [N3928] made `static_assert`'s message parameter optional, making the condition expression automatically usable as an error message.
- In C++20, [P1301R4] introduced an optional message parameter to the `[[nodiscard]]` attribute.
- In C++23, [P2593R1] (adopted through [CWG2518]) allowed `static_assert(false, "message")` to appear in dependent contexts, which effectively turned `static_assert` into a basic compile-time print statement that can be used to emit custom error messages.
- In C++26, [P2573R2] allowed `= delete` to have an optional message parameter.

The key difference between those extensions of the message parameter compared to `static_assert` is that some of them don't generate error messages but allow the library writer to control the content of the warnings generated by the compiler. These extensions have allowed more and more misuse of the library to be diagnosed with user-friendly error messages that directly convey the cause and the fix of errors, and contributed significantly to making C++'s error message more approachable and readable.

However, despite these advancements, one key limitation of the message parameter in all these examples remained: the parameter must be an (unevaluated) string literal. This restriction means that no computation can be performed when providing the custom message, preventing library writers from

providing more useful diagnostic messages to the user. For instance, with the following misuse of `std::format`:

```
std::format("{:d}", "not a number");
// Current error message: (https://godbolt.org/z/enxWj5WY1)
error: call to consteval function 'std::basic_format_string<char, const char*>'
// Potential best we can get using static_assert with fixed message parameter
error: static assertion failed due to requirement [...]: wrong formatter sp
```

The above "potential best" message is already much more user-friendly than the status quo but still lacks some crucial information: the position of the invalid argument (the first, the second, etc.), the actual required type of `:d` (integer), and so on. Let's compare with [Rust's error message](#):

```
format!("{}", 2, "not a number");
// Error message:
error[E0277]: the trait bound `str: Binary` is not satisfied
--> <source>:3:27
   |
3 |     format!("{}", 2, "not a number");
   |                  ---- ~~~~~ the trait `Binary` is not implemented
   |                  |
   |                  required by a bound introduced by this call
   |
= help: the following other types implement trait `Binary`:
      &T
      &mut T
      NonZero<T>
      Saturating<T>
      Wrapping<T>
      i128
      i16
      i32
      and 9 others
[...]
```

The above error message is clearly a step ahead even compared to C++'s "potential best" message achievable. Not only is the precise wrong argument and format specifier pointed out, but several possible fixes (correct type for the argument) are suggested, prompting the user that an integer-like argument is required here.

Such an error message is not possible with `static_assert` with a fixed message parameter. Since no computation is allowed, contextual information, such as the position of the argument, cannot be embedded within the message. In light of this, in the C++17 cycle, [N4433] proposed making `static_assert`'s message parameter dynamically computed and allowing any expression contextually convertible to `const charT*` to appear in the message parameter.

However, the consensus at that time was that the C++ language was not yet ready for dynamically computed string literals, since `constexpr` evaluation was still too limited at that point (for example, `std::string[_view]` could not be used at compile time at that time). Therefore, the first attempt to introduce dynamically generated diagnostic messages to C++ failed.

The `constexpr` evaluation had evolved significantly after that point in time. In C++20, we gained the ability to use `std::string` and `std::string_view` in compile-time, and the extension of allowing user-defined class types in NTPP allowed strings to be passed to functions as compile-time arguments. Nowadays, users no longer need to write compile-time string concatenations on their own, and compile-time string generation facilities such as `constexpr std::format` are also on the horizon (implemented by `{fmt}` already).

Finally, in the C++26 cycle, the matter of user-generated `static_assert` messages is revisited, and [P2741R3] allowed the use of custom-generated string-like parameters as the second message parameter of `static_assert`. This relaxation had allowed for significantly better error messages that embed contextual information to appear in library code, thus allowing the above `std::format` misuse example to output an error message similar to:

```
error: static assertion failed due to requirement [...]: For argument #1, ,
```

Clearly, this new "potential best" error message is much superior to the previous best possible message, thus marking another big step forward for the user-friendliness of C++ error messages. To further expand this advantage, this proposal, therefore, proposes to allow the other three appearances of the message parameter in the core language to also accept user-generated string-like approaches. Such expansion not only unifies the language but also provides more opportunities for more friendly error/warning messages to be provided. Several examples of the possible diagnostics that will be allowed by this proposal can be seen in the Usage Examples section below.

§ 3. Usage Example

§ 3.1. Standard Library

This part contains some concrete examples of how functions and classes in the standard library can benefit from the new flexibility of message parameters. All examples are based on [\[N4988\]](#), and also assumes the existence of `constexpr std::format` ([\[P3391R0\]](#)) and Reflection ([\[P2996R5\]](#)).

One thing to note here is that `[[deprecated]]` is never required on the deprecated features in the standard library. Similarly, `[[nodiscard]]` is regarded as QoI after the adoption of [\[P2422R1\]](#) in St. Louis (2024-06) and no longer appears in the standard library's specification. Therefore, all the following examples only resemble some possible use of the attributes by a standard library vendor. All the examples below that used `[[nodiscard]]` were marked as so in the specification before [\[P2422R1\]](#) landed.

```
// [memory.syn]
namespace std {
    // ...
    template<class T, class... Args> // T is not array
        constexpr unique_ptr<T> make_unique(Args&&... args);
    template<class T> // T is U[]
        constexpr unique_ptr<T> make_unique(size_t n);
    template<class T, class... Args> // T is U[N]
        unspecified make_unique(Args&&...) = delete(format(
            "make_unique<U[{}]>(...) is not supported; perhaps you mean mal
            extent_v<T>, extent_v<T>
        ));

    template<class T, class... Args> // T is not array
        constexpr unique_ptr<T> make_unique_for_overwrite(Args&&... args);
    template<class T> // T is U[]
        constexpr unique_ptr<T> make_unique_for_overwrite(size_t n);
    template<class T, class... Args> // T is U[N]
        unspecified make_unique_for_overwrite(Args&&...) = delete(format(
            "make_unique_for_overwrite<U[{}]>(...) is not supported; perhap
            extent_v<T>, extent_v<T>
        ));
}

// [new.syn]
namespace std {
    // [ptr.laundry], pointer optimization barrier
```

```

template<class T>
[[nodiscard(format(
    "The resulting {}* pointer returned is discarded, which defeats the
    meta::identifier_of(^T)
))]]
constexpr T* launder(T* p) noexcept;
}

// [future.syn]
namespace std {
    // [futures.async], function template async
    template<class F, class... Args>
    [[nodiscard(format(
        "The returning future<{}> is discarded, which makes the given func
        meta::identifier_of(^invoke_result_t<decay_t<F>, decay_t<Args>...>
    ))]]
    future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
    async(F&& f, Args&&... args);
    template<class F, class... Args>
    [[nodiscard(format(
        "The returning future<{}> is discarded, which makes the given func
        meta::identifier_of(^invoke_result_t<decay_t<F>, decay_t<Args>...>
    ))]]
    future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
    async(launch policy, F&& f, Args&&... args);
}

// [depr.relops]
namespace std::rel_ops {
    template<class T>
    [[deprecated(format(
        "operator!= will be automatically generated for class {}",
        meta::identifier_of(^T)
    ))]]
    bool operator!=(const T&, const T&);
}

// [depr.meta.types]
namespace std {
    // ...
    template<size_t Len, size_t Align = default-alignment> // see below
    struct [[deprecated(format(
        "use alignas({}) std::byte[{}] instead.",

```



```

        Align, Len
    )]] aligned_storage;
template<size_t Len, size_t Align = default-alignment > // see below
    using aligned_storage_t = typename aligned_storage<Len, Align>::type;
template<size_t Len, class... Types>
    struct [[deprecated(format(
        "use alignas({}) std::byte[std::max({{}})] instead.",
        string_view{vector{meta::identifier_of(^Types)...} | views::join},
        string_view{vector{format("sizeof({})", meta::identifier_of(^Types)...})}
    )]] aligned_union;
template<size_t Len, class... Types>
    using aligned_union_t = typename aligned_union<Len, Types...>::type;
}

```

Some of the examples above are particularly notable:

- For `std::rel_ops`, the dynamically generated message will embed the corresponding type parameter, which makes it easier for users to pinpoint which class still has relation operators generated by the deprecated `rel_ops` namespace helper.
- For `std::aligned_union`, the dynamically generated message will directly tell users about the correct classes and specific invocations to use:

```

// Usage:
std::aligned_union_t<0, A, B> t_buff;
// Diagnostics:
warning: 'std::aligned_union' is deprecated: use alignas(A, B) std::byte[s]

```

- All of these examples are just a proof of concept on how user-friendly diagnostics is possible; in reality, due to the cost of pulling in the dependency on `std::format` and `<ranges>`, such a message is probably not feasible for most vendors.

§ 3.2. Third-Party Libraries

A potential use case of the new flexibility to third-party library developers is to include the library name in every diagnostic emitted by the types and functions within the library. Although this is already achievable with the current fixed message parameter, adding the name to each message parameter is

error-prone. A better approach is something like (assuming that `std::fixed_string` ([P3094R3]) exists):

```
namespace my_lib
{
    constexpr std::fixed_string lib_name = "myLib";

    [[nodiscard(lib_name + ": api()'s return value should not be discarded)
    void api();

    struct [[deprecated(lib_name + ": class S is deprecated because ...)]
}
```

This design allows the library name to be changed easily by altering `lib_name` and also reduces the possibility of misspelling the library name across different message parameters. Furthermore, a similar design can be applied to functions within a class, with the class name factored out into a `fixed_string` static variable.

§ 4. Design

The general design of this proposal is to allow the following constructs:

```
= delete(message);
[[nodiscard(message)]]
[[deprecated(message)]]
```

where `message` can be a user-provided string-like parameter with the requirements documented below.

§ 4.1. What Is A String?

An immediately obvious question is: What constitute as a string that can be provided as the message parameter? Obviously, we don't want these core language parameters to be tied to a concrete library type such as `std::string_view`.

Fortunately, [P2741R3] already solved this problem for `static_assert`. A compatible string-like type that can be used as the message parameter to `static_assert` has the following requirements:

- Has a `size()` member function that produces a type implicitly convertible to `std::size_t`
- Has a `data()` member function that produces a type implicitly convertible to `const char*`
- The elements in the range `[data(), data()+size())` are valid integer constant expression

This proposal also proposes the same requirement for the message parameter in `= delete`, `[[nodiscard]]`, and `[[deprecated]]`.

§ 4.2. Design Decisions

The following design decisions were made by [\[P2741R3\]](#) and are also proposed by this proposal. Please see [\[P2741R3\]](#) for detailed motivations of these decisions:

- Only contiguous ranges are supported. Thus `data()` and `size()` are used instead of `begin()` and `end()`.
- The message-producing expression is always instantiated but only evaluated if the corresponding declaration fires:

```
template<int N>
void bad() { static_assert(N > 0, "fires"); }
[[nodiscard(bad<0>())]] int fun() { return 2; }
// although fun() is never called, bad<0> is instantiated and the static_a
```

- Only ranges of `char` are supported; no support for `wchar_t`, `char8_t`, `char16_t`, or `char32_t` is proposed following SG16 guidance to [\[P2741R3\]](#).
- Diagnostic encoding is used, and the provided message parameter needs to be converted to that encoding.
- Null-terminated strings are not supported (`\0` will always be printed in the message).

§ 4.3. Attribute Expression Argument

One thing that is novel in this proposal is the fact that arbitrary expression is allowed as arguments to `[[nodiscard]]` and `[[deprecated]]` attributes. This is not completely unheard of, as, in C++23, the `[[assume]]` attribute was adopted via [\[P1774R8\]](#), which also accepts arbitrary expression as an argument. However, the processing of expression in these two cases is different:

- For `[[assume(expr)]]`, `expr` is allowed to be any expression, which is unevaluated but ODR-used (i.e. potentially evaluated).
- For `[[nodiscard(message)]]` and `[[deprecated(message)]]`, `message` must be a constant expression, which is treated as an unevaluated operand if the applied entity is not used.

Here, potentially evaluated and unevaluated operands are complete inversions of each other ([\[basic.def.odr\]/3](#)). The most prominent distinction between those two treatments is the requirement on the definition of the function used:

```
int some_fact(); // no definition
constexpr std::array<char, 5> message(); // no definition yet

// For ODR-use, all used functions must have a definition ([basic.def.odr],
// For unevaluated operand, function definitions are not required at the point of use
template<typename T>
void fun()
{
    [[assume(some_fact() > 42)]]; // IFNDR
    static_assert(false, message()); // okay
}

constexpr std::array<char, 5> message()
{
    return {'H', 'e', 'l', 'l', 'o'};
}

int main()
{
    fun<int>(); // Diagnostics: Hello
}
```

Although as of Oct 2024, the latest released versions of GCC and Clang both do not diagnose the above example ([Compiler Explorer](#)) and instead just silently ignore the assumption.

This can also affect the ABI of a class, as demonstrated in [\[P1774R8\]](#):

```
constexpr auto f(int i) {
    return sizeof( [=] { [[assume(i == 0)]]; } );
}

constexpr auto f2(int i) {
    // nothing is actually captured here
}
```

```

    return sizeof( [=] { static_assert(true, std::array<char, 1>{"Hello"}[s:
}
struct X {
    char data[f(0)];
};
struct X2 {
    char data[f2(0)];
};
// sizeof(X) == 4, sizeof(X2) == 1

```

Overall, the author doesn't think that this discrepancy has that much of an effect since `[[nodiscard]]` and `[[deprecated]]`'s message parameter is required to be a constant expression, and no runtime behavior should be affected.

§ 4.4. Attribute Placement and Name Visibility

An unfortunate consequence of the grammar of attributes is that, as demonstrated in the Usage Example section, the placement of `[[nodiscard]]` and `[[deprecated]]` on class declarations must be after the `class-key`:

```

[[nodiscard(message)]] struct S; // Wrong, attribute ignored
struct [[nodiscard(message)]] S; // Right

```

Fortunately, most compilers will emit a warning that the specified attribute is ignored in this case.

Another problem caused by the placement of attribute and `= delete` is the name visibility problem. Since `= delete` is a kind of definition and attributes must appear before the parameter list, the former can reference the parameters to the function:

```

template<typename T> std::fixed_string message();

template<typename T, typename U>
void fun(const T& t, const U& u) = delete(message<decltype(t + u)>());

template<typename T, typename U>
[[nodiscard(message<decltype(std::declval<T>() + std::declval<U>())>())]] ,
int fun2(const T& t, const U& u);

// Placement:

```

```
[[deprecated]] void fun(); // okay
void fun [[deprecated]] (); // okay, equivalent
void fun() [[deprecated]]; // attribute here applies to the function type,
```

This proposal does not propose to change any grammar of attribute placement, and these unfortunate consequences are only documented here for information.

§ 4.5. What If The Message Parameter Is Invalid?

As discussed before, the message parameter is intended to always be instantiated but only evaluated on triggering:

```
template <auto N>
constexpr std::string_view oups() {
    static_assert(N, "this always fires");
    return "oups!";
}
void f() = delete(oups<false>()); // static_assert fires even though f() is
[[deprecated(oups<false>())]] void f2(); // ditto
```

One possible alternative is to treat this as SFINAE, but that is inconsistent with the behavior of `static_assert` and `[[assume]]` in the function body.

§ 4.6. Alternative Syntax Considered

One possible alternative syntax proposed by [\[P1267R0\]](#) is to use a `[[reason]]` attribute to express the message uniformly. Such a design will look like:

```
[[deprecated, reason(message)]] void fun();
[[nodiscard, reason(message)]] int fun2();
[[reason(message)]] void fun3() = delete;
```

However, `[[deprecated]]`, `[[nodiscard]]`, and `= delete` already accept a message parameter today, so the author thinks this ship has already been sunk.

§ 4.7. Claim of Syntax Space

During the EWGI review of [\[P2573R2\]](#), some people expressed concern about the possibility of that proposal "claiming the syntax space" of possible further enhancement to `= delete`. The same concern may apply to this proposal; namely, it is possible for a future proposal to propose "conditional `[[nodiscard]]`":

```
template<typename T>
[[nodiscard(sizeof(T) > 10)]] int some_fun();
```

While this is certainly a possible future direction, the author doesn't think this proposal claims any syntax space in this regard. Similar to `static_assert`, the only modification needed for conditional `[[nodiscard]]` to still be possible is to require two arguments in that case:

```
template<typename T>
[[nodiscard(sizeof(T) > 10, "reason")]] int some_fun();
```

This does sacrifice some convenience, but if desired, the distinction can be made such that for `[[nodiscard(expr)]]`, if `expr` is convertible to `bool`, then conditional semantic is used; if `expr` satisfies the string-like requirements, the message semantic is used. A tie-breaker between these two is obviously needed for types that satisfy both, but the author doesn't think this is a huge conflict that forms an obstacle to this proposal.

§ 4.8. Interaction with Reflection

Reflection ([\[P2996R5\]](#)) is great and will probably come to C++ in the near future. It will also greatly aid in generating friendly diagnostic messages that display the template type argument's name, as demonstrated in the Usage Examples section above.

However, a natural question arises: should there be a reflection facility to get the message specified to `= delete` and attributes?

In the current reflection proposal ([\[P2996R5\]](#)), there is one metafunction that queries whether the function is deleted: `meta::is_deleted`. No attribute-reflecting mechanisms are proposed in [\[P2996R5\]](#), but there is ongoing work on reflecting attributes first by [\[P1887R1\]](#) and currently by [\[P3385R0\]](#). Should these proposals also propose the ability to query the message provided to the attributes and `= delete`?

The author thinks that they probably should (for instance, by letting `meta::display_name_of` return the message or add a new `meta::message_of`), but this is certainly outside the scope of this proposal. The message parameter already exists in these attributes, and this proposal does not add any new parameters to them, so the difficulty of supporting them should not change drastically.

§ 4.9. Proposal Scope

This proposal is a pure language extension proposal with no library changes involved. It is also a pure addition, so no breaking changes are involved. It is intended practice that exchanging `= delete("message");` for `= delete(func());` will have no breakage and no user-noticeable effect except for the diagnostic messages.

There has been sustained strong opposition to the idea of mandating diagnostic texts in the standard library specifications, and in my opinion, such complaints are justified. This paper does not change this by only proposing to make passing a user-generated string-like object as a message parameter possible and nothing more.

This paper encourages vendors to apply any message they see fit for the purpose as a non-breaking QoI feature.

§ 4.10. Target Vehicle

This proposal targets C++26.

§ 4.11. Feature Test Macro

This proposal bumps the feature test macro for `= delete (__cpp_deleted_function)` and the value reported by `__has_cpp_attribute` for `[[nodiscard]]` and `[[deprecated]]`.

However, one thing to note here is that `__has_cpp_attribute` will report `0` for `[[nodiscard]]` and `[[deprecated]]` if the implementation cannot issue a warning that includes the text of the message provided. This proposal intends for the following rules to be used:

- If an implementation cannot issue warnings that include the message (even a fixed message) at all, `__has_cpp_attribute` should report `0`.
- If an implementation can issue warnings that include a fixed (string literal) message supplied but cannot support arbitrary string-like constant parameter as a message, `__has_cpp_attribute`

should report their current value (201907L for `[[nodiscard]]` and 201309L for `[[deprecated]]` as of Oct 2024).

- If an implementation can issue warnings that include user-generated strings, `__has_cpp_attribute` should report the new values modified by this proposal.

Such rules are consistent with how other feature test macros worked.

§ 5. Implementation Experience

An experimental implementation of the proposed feature is located in my Clang fork at [\[clang-implementation\]](#), which is capable of handling

```
template<std::fixed_string str>
constexpr std::fixed_string<str.size() + 2> fun()
{
    return "^" + str + "$";
}

void fun1() = delete(fun<"Test">());
[[nodiscard(fun<"Test2">())]] int fun2() { return 2; }
[[deprecated(fun<"Test3">())]] void fun3() {}
```

and outputting the following diagnostic messages (close to what the author intended; of course, there are other formats such as put reason on separate lines):

```
propose.cpp:56:5: error: call to deleted function 'fun1': ^Test$
  56 |     fun1(); fun2(); fun3();
      |         ^~~~
propose.cpp:50:6: note: candidate function has been explicitly deleted
  50 | void fun1() = delete(fun<"Test">());
      |     ^
propose.cpp:56:13: warning: ignoring return value of function declared with
  56 |     fun1(); fun2(); fun3();
      |         ^~~~
propose.cpp:56:21: warning: 'fun3' is deprecated: ^Test3$ [-Wdeprecated-decl
  56 |     fun1(); fun2(); fun3();
      |                     ^
propose.cpp:52:3: note: 'fun3' has been explicitly marked deprecated here
```

```
52 | [[deprecated(fun<"Test3">())]] void fun3() {}  
    | ^  
2 warnings and 1 error generated.
```

The implementation is *very* incomplete (no feature-test macro, only a few tests, etc.), so it is only aimed at proving that the vendors can support the proposed feature relatively easily. No significant obstacles are observed during the implementation.

§ 6. Wording

The wording below is based on [\[N4988\]](#).

Wording notes for CWG and editor:

- The wording below renames `static_assert-message` to `diagnostic-message`, which is now used as the grammar for the general diagnostic message parameter.
- The phrase "text of the `diagnostic-message`" is treated as a word of power, and is referenced by other sections defining `= delete` and attributes to simplify the wording.

§ 6.1. 7.7 Constant expressions [expr.const]

[Note 12: Except for a ~~static_assert~~*diagnostic-message*, a manifestly constant-evaluated expression is evaluated even in an unevaluated operand ([expr.context]). — end note]

§ 6.2. 9.1 Preamble [dcl.pre]

Paragraph 1:

Declarations generally specify how names are to be interpreted. Declarations have the form

```
[...]  
static_assertdiagnostic-message:  
    unevaluated-string  
    constant-expression  
static_assert-declaration:
```

```
static_assert ( constant-expression ) ;  
static_assert ( constant-expression , static_assertdiagnostic-message  
[...]
```

Paragraph 12:

If a ~~static_assert~~*diagnostic-message* matches the syntactic requirements of *unevaluated-string*, it is an *unevaluated-string* and the text of the ~~static_assert~~*diagnostic-message* is the text of the *unevaluated-string*. Otherwise, a ~~static_assert~~*diagnostic-message* shall be an expression *M* such that [...]

Paragraph 13:

In a *static_assert-declaration*, the *constant-expression* *E* is contextually converted to `bool` and the converted expression shall be a constant expression ([*expr.const*]). If the value of the expression *E* when so converted is `true` or the expression is evaluated in the context of a template definition, the declaration has no effect and the ~~static_assert~~*diagnostic-message* is an unevaluated operand ([*expr.context*]). Otherwise, the *static_assert-declaration* fails and

- the program is ill-formed, and
- if the ~~static_assert~~*diagnostic-message* is a *constant-expression* *M*,
- *M.size()* shall be a converted constant expression of type `std::size_t` and let *N* denote the value of that expression,
- *M.data()*, implicitly converted to the type "pointer to `const char`", shall be a core constant expression and let *D* denote the converted expression,
- for each *i* where $0 \leq i < N$, *D[i]* shall be an integral constant expression, and
- the text of the ~~static_assert~~*diagnostic-message* is formed by the sequence of *N* code units, starting at *D*, of the ordinary literal encoding ([*lex.charset*]).

Paragraph 14:

Recommended practice: When a *static_assert-declaration* fails, the resulting diagnostic message should include the text of the ~~static_assert~~*diagnostic-message*, if one is supplied.

§ 6.3. 9.5 Function definitions [dcl.fct.def]

§ 6.3.1. 9.5.1 General [dcl.fct.def.general]

Paragraph 1:

Function definitions have the form

```
function-definition:
    attribute-specifier-seqopt decl-specifier-seqopt declarator virt-specifier-seqopt
    attribute-specifier-seqopt decl-specifier-seqopt declarator requires-clauseopt
function-body:
    ctor-initializeropt compound-statement
    function-try-block
    = default ;
    deleted-function-body
deleted-function-body:
    = delete ;
    = delete ( unevaluated-stringdiagnostic-message ) ;
```

§ 6.3.2. 9.5.3 Deleted definitions [dcl.fct.def.delete]

Paragraph 2:

A program that refers to a deleted function implicitly or explicitly, other than to declare it, is ill-formed.

Recommended practice: The resulting diagnostic message should include the text of the *unevaluated-string**diagnostic-message* ([dcl.pre]), if one is supplied.

[*Note 1:* This includes calling the function implicitly or explicitly and forming a pointer or pointer-to-member to the function. It applies even for references in expressions that are not potentially-evaluated. For an overload set, only the function selected by overload resolution is referenced. The implicit odr-use ([basic.def.odr]) of a virtual function does not, by itself, constitute a reference. The *unevaluated-string**diagnostic-message*, if present, can be used to explain the rationale for deletion and/or to suggest an alternative. — *end note*]

§ 6.4. 9.12 Attributes [dcl.attr]

§ 6.4.1. 9.12.5 Deprecated attribute [dcl.attr.deprecated]

Paragraph 1:

The *attribute-token deprecated* can be used to mark names and entities whose use is still allowed, but is discouraged for some reason.

[*Note 1*: In particular, *deprecated* is appropriate for names and entities that are deemed obsolescent or unsafe. — *end note*]

An *attribute-argument-clause* may be present and, if present, it shall have the form:

(~~unevaluated-string~~diagnostic-message)

[*Note 2*: The ~~unevaluated-string~~diagnostic-message in the *attribute-argument-clause* can be used to explain the rationale for deprecation and/or to suggest a replacing entity. — *end note*]

Paragraph 4:

Recommended practice: Implementations should use the *deprecated* attribute to produce a diagnostic message in case the program refers to a name or entity other than to declare it, after a declaration that specifies the attribute. The diagnostic message should include the text of the diagnostic-message ([dcl.pre]) provided within the *attribute-argument-clause* of any *deprecated* attribute applied to the name or entity. The value of a *has-attribute-expression* for the *deprecated* attribute should be $\emptyset$ unless the implementation can issue such diagnostic messages.

§ 6.4.2. 9.12.10 Nodiscard attribute [dcl.attr.nodiscard]

Paragraph 1:

The *attribute-token* `nodiscard` may be applied to a function or a lambda call operator or to the declaration of a class or enumeration. An *attribute-argument-clause* may be present and, if present, it shall have the form:

```
( unevaluated-string diagnostic-message )
```

Paragraph 4:

Recommended practice: Appearance of a `nodiscard` call as a potentially-evaluated discarded-value expression ([`expr.prop`]) is discouraged unless explicitly cast to `void`. Implementations should issue a warning in such cases. The value of a *has-attribute-expression* for the `nodiscard` attribute should be `0` unless the implementation can issue such warnings.

[*Note 2:* This is typically because discarding the return value of a `nodiscard` call has surprising consequences. — *end note*]

The ~~unevaluated-string~~ *text of the diagnostic-message* ([`dcl.pre`]) in a `nodiscard` *attribute-argument-clause* should be used in the message of the warning as the rationale for why the result should not be discarded.

§ 6.5. 15.2 Conditional inclusion [`cpp.cond`]

In [`tab:cpp.cond.ha`], modify the rows as indicated, and substituting `20XXYYL` by the date of adoption.

Attribute	Value
<code>deprecated</code>	201309L <code>20XXYYL</code>
<code>nodiscard</code>	201907L <code>20XXYYL</code>

§ 6.6. 15.11 Predefined macro names [`cpp.predefined`]

In [`tab:cpp.predefined.ft`], modify the rows as indicated, and substituting `20XXYYL` by the date of adoption.

Macro name	Value
<code>__cpp_deleted_function</code>	202403L <code>20XXYYL</code>

§ References

§ Normative References

[CLANG-IMPLEMENTATION]

Yihe Li. *Mick235711's Clang Fork*. URL: <https://github.com/Mick235711/llvm-project/tree/deleted-user-message>

[N4988]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 5 August 2024. URL: <https://wg21.link/n4988>

[P2741R3]

Corentin Jabot. *user-generated static_assert messages*. 16 June 2023. URL: <https://wg21.link/p2741r3>

§ Informative References

[CWG2518]

CWG. *Conformance requirements and #error/#warning*. 13 January 2022. C++23. URL: <https://wg21.link/cwg2518>

[N1720]

R. Klarer, J. Maddock, B. Dawes, H. Hinnant. *Proposal to Add Static Assertions to the Core Language (Revision 3)*. 20 October 2004. URL: <https://wg21.link/n1720>

[N3760]

Alberto Ganesh Barbati. *[[deprecated]] attribute*. 1 September 2013. URL: <https://wg21.link/n3760>

[N3928]

Walter E. Brown. *Extending static_assert, v2*. 14 February 2014. URL: <https://wg21.link/n3928>

[N4433]

Michael Price. *Flexible static_assert messages*. 9 April 2015. URL: <https://wg21.link/n4433>

[P1267R0]

Hana Dusíková, Bryce Adelstein Lelbach. *Custom Constraint Diagnostics*. 8 October 2018. URL: <https://wg21.link/p1267r0>

[P1301R4]

JeanHeyd Meneide, Isabella Muerte. *[[nodiscard("should have a reason")]]*. 5 August 2019. URL: <https://wg21.link/p1301r4>

[P1774R8]

Timur Doumler. *Portable assumptions*. 14 June 2022. URL: <https://wg21.link/p1774r8>

[P1887R1]

Corentin Jabot. *Reflection on attributes*. 13 January 2020. URL: <https://wg21.link/p1887r1>

[P2422R1]

Ville Voutilainen. *Remove nodiscard annotations from the standard library specification*. 28 June 2024. URL: <https://wg21.link/p2422r1>

[P2573R2]

Yihe Li. *= delete("should have a reason");*. 22 March 2024. URL: <https://wg21.link/p2573r2>

[P2593R1]

Barry Revzin. *Allowing static_assert(false)*. 20 January 2023. URL: <https://wg21.link/p2593r1>

[P2996R5]

Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. *Reflection for C++26*. 14 August 2024. URL: <https://wg21.link/p2996r5>

[P3094R3]

Mateusz Pusz. *std::basic_fixed_string*. 20240630. URL: <https://wg21.link/p3094r3>

[P3385R0]

Aurelien Cassagnes, Aurelien Cassagnes, Roman Khoroshikh, Anders Johansson. *Attributes reflection*. 16 September 2024. URL: <https://wg21.link/p3385r0>

[P3391R0]

Barry Revzin. *constexpr std::format*. 12 September 2024. URL: <https://wg21.link/p3391r0>